

课程作业二

补全 ViT 代码、目标检测评价指标计算、YOLO 模型推理

常雅静 23307130470

2026 年 4 月 30 日

目录

1	总述	3
2	实验一：Vision Transformer (ViT)	3
2.1	实验目的	3
2.2	模型结构	3
2.2.1	Patch Embedding	4
2.2.2	Position Encoding	4
2.2.3	Transformer Encoder	4
2.2.4	分类头 (Head)	5
2.3	实验设置	5
2.3.1	数据集	5
2.3.2	Baseline 参数设置	5
2.4	实验结果	5
2.4.1	Bad Cases 分析	5
2.4.2	Baseline 及参数消融实验	6
2.5	结果分析与深入探讨	7
2.6	综合分析	8

2.6.1	最优参数组合与讨论	8
2.6.2	实验结论	9
3	实验二：目标检测评价指标计算	9
3.1	实验参数说明	9
3.1.1	真实框 (Ground Truth, GT)	9
3.1.2	预测框 (Prediction)	10
3.1.3	评价规则	10
3.2	任务 1: IoU 计算	10
3.3	任务 2/3/4: 指标判断与计算	11
3.3.1	TP / FP 判定	11
3.3.2	FN 判定	12
3.3.3	判定结果表格	12
3.3.4	Precision 与 Recall 计算	12
4	实验三：YOLO 模型推理	12
4.1	实验参数	12
4.2	检测结果数据	13
4.3	结果可视化与分析	13
A	附录：完整 Python 代码实现	14
A.1	实验一：ViT 完整代码	14
A.2	实验三：YOLO 目标推理代码	17

1 综述

本次课程作业涵盖了计算机视觉领域中从前沿模型实现、基础评价指标计算到先进算法部署推理的三个核心环节。具体而言：

实验一：Vision Transformer (ViT) 的实现与分析。该部分聚焦于图像分类任务，基于 PyTorch 补全了 ViT 的核心代码（包括图像分块与 Transformer 编码器），并在 MNIST 数据集上验证了模型的有效性。通过系统性的消融实验，深入探讨了 Patch 大小、特征维度、网络层数等超参数对模型分类性能的具体影响。

实验二：目标检测评价指标的计算。该部分回归目标检测的基础理论，通过具体的数据实例，逐步推导并计算了交并比 (IoU)、真阳性 (TP)、假阳性 (FP) 及假阴性 (FN) 等关键中间变量，并最终求得了精确率 (Precision) 与召回率 (Recall)，有效加深了对目标检测性能评价体系的底层理解。

实验三：YOLO 模型推理与可视化。该部分侧重于算法的工程实践，利用业界广泛采用的 YOLO 预训练模型对真实图像进行目标检测。结合置信度阈值过滤，并利用 OpenCV 完成了预测框与标签的绘制，实现了检测结果的直观可视化。

这三个实验将理论推导、代码编写与模型部署紧密结合，系统性地巩固了对现代计算机视觉深度学习算法框架及其评估流程的掌握。

2 实验一：Vision Transformer (ViT)

2.1 实验目的

本实验基于 PyTorch 实现 Vision Transformer (ViT)，在 MNIST 数据集上进行图像分类实验，主要目标包括：

- 补全 ViT 模型核心代码 (Patch Embedding + Transformer Encoder)
- 理解 ViT 的整体结构与数据流
- 分析关键超参数对模型性能的影响
- 结合错误样本 (bad cases) 分析模型分类边界与鲁棒性

2.2 模型结构

ViT 的整体流程如下：

Image $\rightarrow$ Patch Embedding $\rightarrow$ Token Sequence $\rightarrow$ Position Encoding
 $\rightarrow$ Transformer Encoder $\rightarrow$ Mean Pooling $\rightarrow$ MLP Head $\rightarrow$ Prediction

与传统 CNN 使用卷积逐层提取局部特征不同，ViT 将图像视为序列数据处理，利用自注意力机制直接建模全局区域之间的关系，因此具有较强的全局感知能力。

2.2.1 Patch Embedding

输入图像大小为 28×28 ，将其划分为若干固定大小 patch。实验中通过卷积实现：

```
nn.Conv2d(kernel_size=patch_size, stride=patch_size)
```

其本质等价于：将图像切分为不重叠 patch、每个 patch 展平、线性映射到 embedding 空间。例如在本实验中：

- patch_size=4：产生 $7 \times 7 = 49$ 个 token
- patch_size=7：产生 $4 \times 4 = 16$ 个 token
- patch_size=14：产生 $2 \times 2 = 4$ 个 token

patch 数量越多，序列越长，注意力计算量越大。

2.2.2 Position Encoding

由于 Transformer 本身不具备空间位置信息，因此引入可学习位置编码：

$$x = x + \text{pos_embed}$$

用于表示 patch 在图像中的相对位置，使模型能够区分“左上角笔画”与“右下角笔画”。

2.2.3 Transformer Encoder

实验采用 PyTorch 标准编码器堆叠而成，每层包含：

- Multi-Head Self Attention
- Feed Forward Network
- Residual Connection

- Layer Normalization

其作用是让每个 patch 与其他所有 patch 建立联系，从而学习数字整体结构信息。例如识别数字 8 时，上下两个圆环结构可以同时被建模。

2.2.4 分类头 (Head)

标准 ViT 常使用 CLS Token 进行分类，本实验采用更简洁的 mean pooling：

$$x = \text{mean}(x, \text{dim} = 1)$$

即对所有 token 特征取平均，得到全局图像表示，再输入全连接层输出 10 类概率。该方法参数更少，在 MNIST 小规模任务上表现稳定。

2.3 实验设置

2.3.1 数据集

采用 MNIST 手写数字数据集，含 60000 张训练样本、10000 张测试样本；图像分辨率 28×28 ，共分为 10 个类别（数字 0-9）。

2.3.2 Baseline 参数设置

表 1: Baseline 模型参数设置

参数名	值	训练设置	值
patch_size	7	Optimizer	Adam
embed_dim	64	Learning Rate	1×10^{-3}
num_layers	2	Epochs	2
nhead	2	Batch Size	32

所有实验重复运行 3 次，取平均 Accuracy。

2.4 实验结果

2.4.1 Bad Cases 分析

从错误样本可以发现，模型误分类主要集中在以下几类情况：

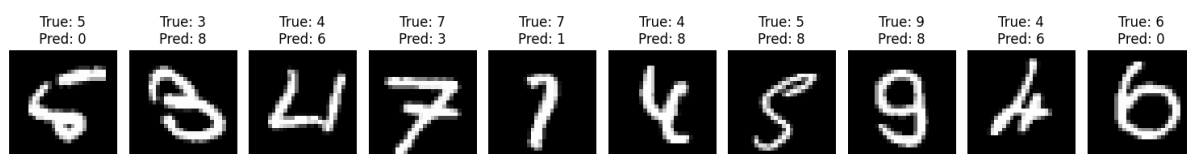


图 1: Bad Cases (错误分类样本)

- **数字形态相近**: 如 3 与 8, 7 与 1 等, 在手写体中边界较模糊, 容易混淆。
- **书写过于潦草**: 某些数字笔画断裂、倾斜严重或闭合不完整, 使得结构特征缺失。
- **局部噪声干扰**: 个别图存在粗细不均、位置偏移等问题, 影响 patch 表征稳定性。
- **低分辨率限制**: MNIST 图像仅 28×28 , 部分细节在大 patch 划分时被压缩, 导致局部信息损失。

说明即使 ViT 具备全局建模能力, 在边界模糊样本上仍依赖清晰结构信息。

2.4.2 Baseline 及参数消融实验

表 2: 各项超参数对模型准确率 (Accuracy) 的影响

实验组	参数修改	Accuracy
Baseline	默认参数	94.68%
Patch Size	Patch Size = 4	94.34%
	Patch Size = 7 (Baseline)	94.68%
	Patch Size = 14	96.56%
Embed Dim	Embed Dim = 32	94.35%
	Embed Dim = 64 (Baseline)	94.68%
	Embed Dim = 128	94.84%
Num Layers	Num Layers = 1	93.57%
	Num Layers = 2 (Baseline)	94.68%
	Num Layers = 3	95.47%
N Head	N Head = 1	94.15%
	N Head = 2 (Baseline)	94.68%
	N Head = 4	94.88%

2.5 结果分析与深入探讨

- **Patch Size 对模型性能影响最显著。**实验结果表明：

$$14 > 7 > 4$$

说明在 MNIST 这种结构简单、背景干净、目标居中的任务中，较大的 patch 更具优势。其原因在于手写数字识别更依赖整体轮廓信息，而非局部纹理细节。当 patch_size=14 时，图像仅被划分为

$$\left(\frac{28}{14}\right)^2 = 4$$

个 patch，每个 patch 可覆盖更大区域，从而直接保留数字主体结构，例如数字 0 的闭环、数字 7 的斜线结构等。

同时，token 数量减少后，自注意力计算复杂度显著下降。由于注意力复杂度近似为： $O(N^2)$ ，其中 N 为 token 数，因此较大的 patch size 能降低计算负担，提高训练稳定性，并减少过拟合风险。

相比之下，patch_size=4 会产生较长序列，模型需处理更多局部冗余信息，在仅训练 2 个 epoch 的条件下难以充分优化，因此准确率最低。

- **Embedding Dim 增大可提升表示能力，但收益有限。**

实验结果为：

$$128 > 64 > 32$$

说明增大 embedding 维度对性能有正向作用。Embedding Dim 决定每个 token 的特征表示容量，维度越高，模型越容易表达复杂结构信息，如笔画粗细、曲线变化、闭环完整性及局部区域组合关系。

当维度从 32 提升至 64 时，模型表示能力明显增强；继续提升至 128 时仍有小幅提升，说明高维空间有助于学习更精细的类别边界。

但由于 MNIST 图像内容简单，64 维特征已能覆盖大部分有效信息，因此继续增大维度的收益有限。同时更高维度会带来更大的参数量、显存占用和训练时间，因此在轻量任务中需兼顾效率。

- **增加 Transformer 层数对准确率提升明显。**

实验结果为：

$$3 > 2 > 1$$

说明增加网络深度是提升性能的有效方式之一。Transformer 每增加一层，本质上都会执行一次全局特征交互与非线性映射，使模型逐步形成更高层次语义表示，

即：

局部笔画 $\rightarrow$ 几何结构 $\rightarrow$ 完整数字类别

例如识别数字 8 时，浅层模型只能关注局部弧线，而深层模型能够综合上下两个闭环结构进行判断。

实验中层数从 1 增加到 3，带来的提升幅度明显高于 embedding dim 的提升，说明对于该任务而言，增加特征交互深度比单纯扩大特征宽度更有效。

- 多头注意力机制提升稳定，但增益相对有限。

实验结果为：

$$4 > 2 > 1$$

说明增加注意力头数能够持续提升性能，但提升幅度小于 Patch Size 与网络深度。

多头注意力机制可理解为多个并行观察视角，不同 head 能关注不同结构信息，例如竖直笔画、闭环区域、斜线连接关系以及上下空间结构等，因此头数增加后，模型特征表达更加丰富，鲁棒性更强。

但由于 MNIST 数据集类别边界清晰、背景简单、分辨率较低，不需要过多复杂注意力模式即可完成分类，因此头数增加带来的收益相对有限。在更复杂的自然图像任务中，多头机制通常会发挥更大作用。

2.6 综合分析

根据实验结果，各参数影响程度可总结为：

$$\text{Patch Size} > \text{Num Layers} > \text{N Head} > \text{Embed Dim}$$

说明在轻量视觉任务中，决定 token 序列长度与信息粒度的 patch 划分方式，比单纯增加模型宽度更重要。

同时说明：

- 小数据集任务优先优化输入表示方式；
- 中等深度 Transformer 即可获得较好效果；
- 盲目增大维度收益有限。

2.6.1 最优参数组合与讨论

根据消融实验结果，组合得到最优配置：

表 3: 最优参数组合

Patch Size	Embed Dim	Num Layers	N Head	Epochs	Accuracy
14	128	3	4	2	96.85%

相比 Baseline (94.68%), 提升:

$$96.85\% - 94.68\% = 2.17\%$$

说明合理的结构设计可显著提升性能, 而无需大幅增加训练轮数。

2.6.2 实验结论

本实验完成了 Vision Transformer 在 MNIST 上的实现与系统分析, 得到如下结论:

- ViT 即使在简单视觉任务中也具有良好分类能力;
- Patch Size 是最关键超参数;
- 增加网络深度比单纯增大宽度更有效;
- 多头注意力能增强特征表达鲁棒性;
- 错误样本主要来自数字结构相似与书写模糊情况。

总体而言, ViT 已验证其在图像分类任务中的有效性。

3 实验二：目标检测评价指标计算

3.1 实验参数说明

3.1.1 真实框 (Ground Truth, GT)

坐标格式: (x_1, y_1, x_2, y_2)

- **GT1** = (50, 50, 150, 200)
- **GT2** = (220, 60, 320, 210)

3.1.2 预测框 (Prediction)

- **P1** = (48, 52, 152, 198), score = 0.95
- **P2** = (60, 70, 140, 180), score = 0.80
- **P3** = (215, 65, 315, 205), score = 0.90
- **P4** = (340, 80, 390, 180), score = 0.70

3.1.3 评价规则

1. 类别均预测为 person。
2. $\text{IoU} \geq 0.5$: 预测框与真实框成功匹配。
3. 每个真实框最多匹配一个预测框。
4. 同一真实框对应多个预测框时, 仅保留置信度最高的为 TP, 其余为 FP。

3.2 任务 1: IoU 计算

IoU 定义: 交并比 (Intersection over Union) 是预测框与真实框的交集面积除以并集面积:

$$\text{IoU} = \frac{\text{交集面积}}{\text{预测框面积} + \text{真实框面积} - \text{交集面积}}$$

逐框详细计算过程 (保留三位小数):

- **P1 vs GT1**

$$\text{交集宽} = \min(152, 150) - \max(48, 50) = 150 - 50 = 100$$

$$\text{交集高} = \min(198, 200) - \max(52, 50) = 198 - 52 = 146$$

$$\text{交集面积} = 100 \times 146 = 14600$$

$$\text{P1 面积} = (152 - 48) \times (198 - 52) = 104 \times 146 = 15184$$

$$\text{GT1 面积} = (150 - 50) \times (200 - 50) = 100 \times 150 = 15000$$

$$\text{并集面积} = 15184 + 15000 - 14600 = 15584$$

$$\text{IoU} = 14600 / 15584 \approx \mathbf{0.937}$$

- **P2 vs GT1**

$$\text{交集宽} = \min(140, 150) - \max(60, 50) = 140 - 60 = 80$$

$$\text{交集高} = \min(180, 200) - \max(70, 50) = 180 - 70 = 110$$

$$\text{交集面积} = 80 \times 110 = 8800$$

$$\text{P2 面积} = (140 - 60) \times (180 - 70) = 80 \times 110 = 8800$$

$$\text{并集面积} = 8800 + 15000 - 8800 = 15000$$

$$\text{IoU} = 8800/15000 \approx \mathbf{0.587}$$

• P3 vs GT2

$$\text{交集宽} = \min(315, 320) - \max(215, 220) = 315 - 220 = 95$$

$$\text{交集高} = \min(205, 210) - \max(65, 60) = 205 - 65 = 140$$

$$\text{交集面积} = 95 \times 140 = 13300$$

$$\text{P3 面积} = (315 - 215) \times (205 - 65) = 100 \times 140 = 14000$$

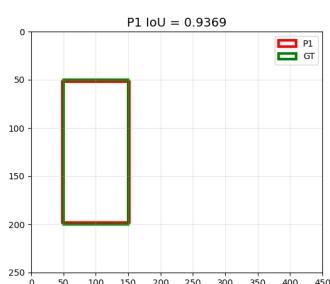
$$\text{GT2 面积} = (320 - 220) \times (210 - 60) = 100 \times 150 = 15000$$

$$\text{并集面积} = 14000 + 15000 - 13300 = 15700$$

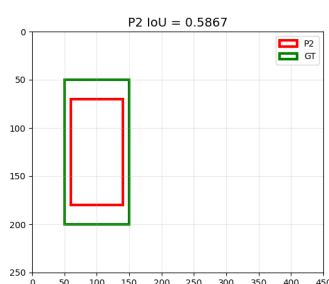
$$\text{IoU} = 13300/15700 \approx \mathbf{0.847}$$

• P4 与所有 GT

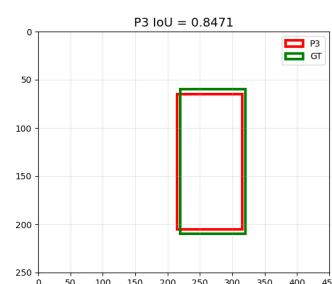
在空间上均无交集, $\text{IoU} = 0$



(a) P1 vs GT1



(b) P2 vs GT1



(c) P3 vs GT2

图 2: Three IoU Visualization Results

3.3 任务 2/3/4: 指标判断与计算

3.3.1 TP / FP 判定

TP (True Positive): 真正例, 即预测框与真实框匹配成功 ($\text{IoU} \geq 0.5$) 且置信度最高。本题中 P1 成功匹配 GT1, P3 成功匹配 GT2, 且均未被更早的预测框占用, 故 $\text{TP} = 2$ 。

FP (False Positive): 假正例, 即预测框未能与任何真实框成功匹配, 或因重复匹配同一真实框而被舍弃。本题中 P2 虽满足 IoU 条件但 GT1 已被 P1 占用, P4 则因 IoU 为 0 未能匹配, 故 $\text{FP} = 2$ 。

依据置信度降序排序: **P1(0.95) > P3(0.90) > P2(0.80) > P4(0.70)**

- **P1:** 匹配 GT1, 且 $\text{IoU} \approx 0.937 \geq 0.5 \rightarrow \text{TP}$ (此时 GT1 被标记为已占用)。

- **P3**: 匹配 GT2, 且 $\text{IoU} \approx 0.847 \geq 0.5 \rightarrow \text{TP}$ (此时 GT2 被标记为已占用)。
- **P2**: 与 GT1 的 $\text{IoU} \approx 0.587 \geq 0.5$, 但对应 GT1 已被置信度更高的 P1 占用, 根据“每个真实框最多匹配一个预测框”的原则, 匹配失败 $\rightarrow \text{FP}$ 。
- **P4**: 与任何真实框均无交集, 无匹配目标 $\rightarrow \text{FP}$ 。

3.3.2 FN 判定

FN (False Negative): 假阴性, 即未被成功匹配的真实框。因为本题中仅有的真实框 GT1、GT2 均已成功被最高置信度的预测框匹配, 故不存在漏检, $\text{FN} = 0$ 。

3.3.3 判定结果表格

表 4: 预测结果综合判定表

预测框	置信度 (排序)	IoU	匹配 GT (是否唯一)	最终结果
P1	0.95 (1)	0.937	匹配 GT1 (是)	TP
P3	0.90 (2)	0.847	匹配 GT2 (是)	TP
P2	0.80 (3)	0.587	匹配 GT1 (否, 被 P1 占用)	FP
P4	0.70 (4)	0	无匹配 (/)	FP

3.3.4 Precision 与 Recall 计算

Precision (精确率): 衡量预测结果中“抓得准”的比例, Recall (召回率): 衡量真实目标中“找得全”的比例。由上述判定可知各项统计值为: $\text{TP} = 2, \text{FP} = 2, \text{FN} = 0$ 。

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{2}{2 + 2} = 0.5$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{2}{2 + 0} = 1.0$$

4 实验三: YOLO 模型推理

4.1 实验参数

- **模型**: YOLO26m 预训练模型
- **输入图像**: 测试图片 cat-and-dog.jpg, 常规分辨率

- 置信度阈值: $\text{conf} = 0.25$
- 输出结果: 带检测框的标注图像

4.2 检测结果数据

- 检测类别: 16 类别名称: dog 检测置信度: 0.9187126159667969
边角坐标: [270.72, 167.07, 1375.80, 1141.31]
- 检测类别: 15 类别名称: cat 检测置信度: 0.3004116714000702
边角坐标: [589.62, 609.67, 1042.20, 1139.08]

4.3 结果可视化与分析



图 3: cat and dog 原图

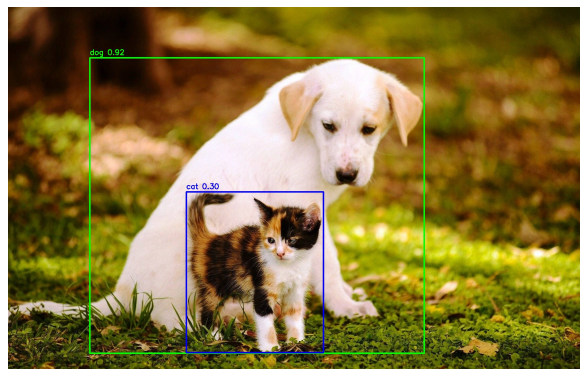


图 4: YOLO 模型目标检测结果图

可视化说明: 利用 OpenCV 进行标注时, 为不同类别分配了不同颜色的边框 (如 cat 为蓝色, dog 为绿色), 设定线宽为 3, 并使用 OpenCV 默认字体在框上方标注名称及置信度数值。

问题分析: 根据检测结果可以看出, 图中 dog 的置信度高达 0.92, 而 cat 的置信度仅为 0.30。cat 的置信度十分接近设定的阈值 0.25, 说明模型对猫的特征提取不够确信。可能的原因包括: 猫的后脚在原图中存在较严重的遮挡, 且该猫的毛色较为特殊, 与一只眼睛几乎融为一体。如果实际应用中将阈值调高 (例如 0.5), 则该目标极易出现漏检 (FN), 这体现了阈值选取在精度和召回率间的权衡。

A 附录：完整 Python 代码实现

A.1 实验一：ViT 完整代码

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.utils.data import DataLoader
5 from torchvision import datasets, transforms
6 import matplotlib.pyplot as plt
7
8 # =====
9 # 1. 超参数 (简单易懂)
10 # =====
11 batch_size = 32
12 lr = 1e-3
13 epochs = 2
14 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
15
16 # =====
17 # 2. MNIST 数据加载
18 # =====
19 transform = transforms.Compose([
20     transforms.ToTensor(),
21     transforms.Normalize((0.1307,), (0.3081,))
22 ])
23
24 train_dataset = datasets.MNIST('./data', train=True, download=True, transform=transform)
25 test_dataset = datasets.MNIST('./data', train=False, download=True, transform=transform)
26
27 train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
28 test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
29
30 # =====
31 # 3. 极简 ViT 模型 (核心: 分 Patch + 编码 + Transformer)
32 # =====
33 class SimpleViT(nn.Module):
34     def __init__(self, img_size=28, patch_size=7, in_chans=1, num_classes=10,
35                 embed_dim=64, num_layers=2, nhead=2, dim_feedforward=128):
36         super().__init__()
37         self.patch_size = patch_size
38         self.num_patches = (img_size // patch_size) ** 2 # 28/7=4 -> 4*4=16 个 patch
39
40         # 关键步骤1: 分 Patch + 编码 (卷积一步实现)
41         # 以下为本次作业补全代码
42         self.patch_embed = nn.Conv2d(
43             in_channels=in_chans,
44             out_channels=embed_dim,
45             kernel_size=patch_size,
46             stride=patch_size
47         )
48
49         # 关键步骤2: 位置编码
50         # 以下为本次作业补全代码
51         self.pos_embed = nn.Parameter(torch.zeros(1, self.num_patches, embed_dim))
52         self.encoder_layer = nn.TransformerEncoderLayer(
```

```

52         d_model=embed_dim,
53         nhead=nhead,
54         batch_first=True,
55         dim_feedforward=dim_feedforward
56     )
57     self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
58     # 分类头
59     self.head = nn.Linear(embed_dim, num_classes)
60
61     def forward(self, x):
62         # -----
63         # 核心：图像 -> 分块 -> 序列
64         # -----
65         # 以下为本次作业补全代码
66         x = self.patch_embed(x) # (B, embed_dim, H_patch, W_patch)
67         x = x.flatten(2)        # (B, embed_dim, num_patches)
68         x = x.transpose(1, 2)   # (B, num_patches, embed_dim)
69
70         # 加位置信息
71         # 以下为本次作业补全代码
72         x = x + self.pos_embed
73
74         # Transformer 推理
75         x = self.transformer(x)
76         x = x.mean(dim=1)      # 对序列维度做平均聚合
77
78         # 分类输出
79         return self.head(x)
80
81     # =====
82     # 4. 初始化模型、损失、优化器
83     # =====
84     model = SimpleViT().to(device)
85     criterion = nn.CrossEntropyLoss()
86     optimizer = optim.Adam(model.parameters(), lr=lr)
87
88     # =====
89     # 5. 训练代码
90     # =====
91     def train(model, loader, criterion, optimizer, device):
92         model.train()
93         total_loss = 0
94         for data, target in loader:
95             data, target = data.to(device), target.to(device)
96             optimizer.zero_grad()
97             output = model(data)
98             loss = criterion(output, target)
99             loss.backward()
100            optimizer.step()
101            total_loss += loss.item()
102            print(f"训练平均损失: {total_loss / len(loader):.4f}")
103
104     # =====
105     # 6. 测试代码 (收集 bad case)
106     # =====
107     def test(model, loader, device):
108         model.eval()
109         correct = 0

```

```

110     bad_cases = []
111     with torch.no_grad():
112         for data, target in loader:
113             data, target = data.to(device), target.to(device)
114             output = model(data)
115             pred = output.argmax(dim=1)
116             correct += pred.eq(target).sum().item()
117             wrong_indices = (pred != target).nonzero(as_tuple=True)[0]
118             for idx in wrong_indices:
119                 bad_cases.append((data[idx].cpu(), target[idx].cpu(), pred[idx].cpu()))
120     acc = 100 * correct / len(loader.dataset)
121     print(f"测试准确率: {acc:.2f}%\n")
122     return bad_cases
123
124     # =====
125     # 7. 可视化 bad case
126     # =====
127     def visualize_bad_cases(bad_cases, num_cases=10):
128         if len(bad_cases) == 0:
129             print("没有 bad case! ")
130             return
131         num_cases = min(num_cases, len(bad_cases))
132         fig, axes = plt.subplots(1, num_cases, figsize=(15, 3))
133         if num_cases == 1:
134             axes = [axes]
135         for i in range(num_cases):
136             img, true_label, pred_label = bad_cases[i]
137             img = img.squeeze().numpy() # 移除通道维度
138             axes[i].imshow(img, cmap='gray')
139             axes[i].set_title(f'True: {true_label}\nPred: {pred_label}')
140             axes[i].axis('off')
141         plt.tight_layout()
142         plt.savefig('bad_cases.png')
143         print(f"Bad cases saved to bad_cases.png")
144
145     def run_experiment(config):
146         print(f"\n===== 运行配置: {config['name']} =====")
147         model = SimpleViT(
148             patch_size=config['patch_size'],
149             embed_dim=config['embed_dim'],
150             num_layers=config['num_layers'],
151             nhead=config['nhead']
152         ).to(device)
153         criterion = nn.CrossEntropyLoss()
154         optimizer = optim.Adam(model.parameters(), lr=lr)
155
156         for epoch in range(1, epochs + 1):
157             print(f"==== Epoch {epoch}/{epochs} ====")
158             train(model, train_loader, criterion, optimizer, device)
159             bad_cases = test(model, test_loader, device)
160             return bad_cases
161
162     def main():
163         configs = [
164             {'name': 'baseline', 'patch_size': 7, 'embed_dim': 64, 'num_layers': 2, 'nhead': 2},
165             {'name': 'patch_size=14', 'patch_size': 14, 'embed_dim': 64, 'num_layers': 2, 'nhead': 2},
166             {'name': 'patch_size=4', 'patch_size': 4, 'embed_dim': 64, 'num_layers': 2, 'nhead': 2}

```



```
2},
167     {'name': 'embed_dim=32', 'patch_size': 7, 'embed_dim': 32, 'num_layers': 2, 'nhead':
2},
168     {'name': 'embed_dim=128', 'patch_size': 7, 'embed_dim': 128, 'num_layers': 2, 'nhead':
2},
169     {'name': 'num_layers=1', 'patch_size': 7, 'embed_dim': 64, 'num_layers': 1, 'nhead':
2},
170     {'name': 'num_layers=3', 'patch_size': 7, 'embed_dim': 64, 'num_layers': 3, 'nhead':
2},
171     {'name': 'nhead=1', 'patch_size': 7, 'embed_dim': 64, 'num_layers': 2, 'nhead': 1},
172     {'name': 'nhead=4', 'patch_size': 7, 'embed_dim': 64, 'num_layers': 2, 'nhead': 4},
173 ]
174
175 results = {}
176 for cfg in configs:
177     bad_cases = run_experiment(cfg)
178     acc = 100 - 100 * len(bad_cases) / len(test_loader.dataset)
179     results[cfg['name']] = acc
180
181 print("\n===== 实验结果摘要 =====")
182 for name, acc in results.items():
183     print(f"{name}: {acc:.2f}%")
184
185 print("\n===== 可视化最后一个配置的 bad case =====")
186 visualize_bad_cases(bad_cases)
187
188 if __name__ == '__main__':
189     main()
```

Listing 1: ViT PyTorch 实现

A.2 实验三: YOLO 目标推理代码

```
1 from ultralytics import YOLO
2 import cv2
3
4 # 运行环境: ultralytics 8.0.x, OpenCV 4.8.x
5 # 使用 YOLO26 end2end 模式 (无 NMS)
6 # 加载模型
7 model = YOLO("yolo26m.pt")
8
9 # 读取图片
10 img = cv2.imread("cat-and-dog.jpg")
11
12 # 设定置信度阈值进行推理
13 results = model(img, conf=0.25)
14
15 # 每个类别颜色 (BGR格式)
16 colors = {
17     "cat": (255, 0, 0),      # 蓝
18     "dog": (0, 255, 0),     # 绿
19     "person": (0, 0, 255),  # 红
20     "car": (0, 255, 255),   # 黄
21 }
22
```

```
23 for r in results:
24     for box in r.bboxes:
25         cls = int(box.cls[0])
26         conf = float(box.conf[0])
27         name = model.names[cls]
28
29         x1, y1, x2, y2 = map(int, box.xyxy[0])
30
31         # 若类别没设颜色, 默认白色
32         color = colors.get(name, (255, 255, 255))
33
34         # 画框
35         cv2.rectangle(img, (x1, y1), (x2, y2), color, 3)
36
37         # 写标签
38         cv2.putText(
39             img,
40             f"{name} {conf:.2f}",
41             (x1, y1 - 10),
42             cv2.FONT_HERSHEY_SIMPLEX,
43             0.8,
44             color,
45             2
46         )
47
48     # 保存推理结果
49     cv2.imwrite("result_color.jpg", img)
50     print("已保存 result_color.jpg")
```

Listing 2: YOLO 目标推理脚本